

HDOC DAY-34

Title: Verification of Distributive Properties of Matrix Multiplication over Matrix Addition

1. Problem Statement

Write a C program to perform the following menu-driven operations using separate functions:

1. Read three variable-length 2D arrays and verify the left distributive property: $A \times (B + C) = (A \times B) + (A \times C)$. The dimensions of input matrices are given as follows: $A (m \times n)$, $B (n \times p)$, $C (n \times p)$.
2. Read three variable-length 2D arrays and verify the right distributive property: $(A + B) \times C = (A \times C) + (B \times C)$. The dimensions of input matrices are given as follows: $A (m \times n)$, $B (m \times n)$, $C (n \times p)$.
3. Exit.

Note: The matrices obtained on both the left-hand side and the right-hand side of each distributive property will have dimensions $(m \times p)$. Display both resulting matrices and state whether the selected distributive property is verified.

2. Theoretical Background & Mathematical Proof

1. Left Distributive Property:

Given $A \in \mathbb{R}^{m \times n}$, $B \in \mathbb{R}^{n \times p}$, and $C \in \mathbb{R}^{n \times p}$:

$$\begin{aligned}[A(B + C)]_{ij} &= \sum_{k=1}^n a_{ik} (b_{kj} + c_{kj}) = \sum_{k=1}^n (a_{ik} b_{kj} + a_{ik} c_{kj}) \\ &= \sum_{k=1}^n a_{ik} b_{kj} + \sum_{k=1}^n a_{ik} c_{kj} = [AB + AC]_{ij}\end{aligned}$$

Both resulting matrices have dimensions $(m \times p)$ and identical values.

2. Right Distributive Property:

Given $A \in \mathbb{R}^{m \times n}$, $B \in \mathbb{R}^{m \times n}$, and $C \in \mathbb{R}^{n \times p}$:

$$\begin{aligned}[(A + B)C]_{ij} &= \sum_{k=1}^n (a_{ik} + b_{ik}) c_{kj} = \sum_{k=1}^n (a_{ik} c_{kj} + b_{ik} c_{kj}) \\ &= \sum_{k=1}^n a_{ik} c_{kj} + \sum_{k=1}^n b_{ik} c_{kj} = [AC + BC]_{ij}\end{aligned}$$

Both resulting matrices have dimensions $(m \times p)$ and identical values.

3. Test Case Table

Test Case	Option	Input Dimensions & Matrices	LHS Result (m×p) JPG	RHS Result (m×p) JPG	Status
1	Left Distributive	$m = 2, n = 2, p = 2$			

$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, B = \begin{bmatrix} 2 & 0 \\ 1 & 3 \end{bmatrix}, C = \begin{bmatrix} 1 & 1 \\ 0 & 2 \end{bmatrix} \mid \begin{bmatrix} 7 & 7 \\ 19 & 19 \end{bmatrix} \mid \begin{bmatrix} 7 & 7 \\ 19 & 19 \end{bmatrix}$
 | Verified | | **2** | Right Distributive | $m = 2, n = 2, p = 2$

$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, B = \begin{bmatrix} 2 & 1 \\ 0 & 2 \end{bmatrix}, C = \begin{bmatrix} 1 & 2 \\ 3 & 0 \end{bmatrix} \mid \begin{bmatrix} 12 & 6 \\ 18 & 12 \end{bmatrix} \mid \begin{bmatrix} 12 & 6 \\ 18 & 12 \end{bmatrix}$
 | Verified | | **3** | Exit | Choice 3 | System Exit | Terminated cleanly | Pass |

4. Source Code Listing

```

#include
#include

void read_matrix(int rows, int cols, int mat[rows][cols], const char
*name) {
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
  
```

```

        scanf("%d", &mat[i][j]);
    }

void print_matrix(int rows, int cols, int mat[rows][cols]) {
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) printf("%d\t", mat[i][j]);
        printf("\n");
    }
}

bool are_matrices_equal(int rows, int cols, int mat1[rows][cols], int
mat2[rows][cols]) {
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            if (mat1[i][j] != mat2[i][j]) return false;
    return true;
}

void add_matrices(int rows, int cols, int A[rows][cols], int B[rows][cols],
int res[rows][cols]) {
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            res[i][j] = A[i][j] + B[i][j];
}

void multiply_matrices(int r1, int c1, int A[r1][c1], int r2, int c2, int
B[r2][c2], int res[r1][c2]) {
    for (int i = 0; i < r1; i++) {
        for (int j = 0; j < c2; j++) {
            res[i][j] = 0;
            for (int k = 0; k < c1; k++) res[i][j] += A[i][k] * B[k][j];
        }
    }
}

void verify_left_distributive(void) {
    int m, n, p;
    printf("Enter m, n, p: "); scanf("%d %d %d", &m, &n, &p);
    int A[m][n], B[n][p], C[n][p], B_plus_C[n][p], LHS[m][p], AB[m][p],
AC[m][p], RHS[m][p];

```

```
    read_matrix(m, n, A, "A"); read_matrix(n, p, B, "B"); read_matrix(n, p, C, "C");
```

```
    add_matrices(n, p, B, C, B_plus_C); multiply_matrices(m, n, A, n, p, B_plus_C, LHS);  
    multiply_matrices(m, n, A, n, p, B, AB); multiply_matrices(m, n, A, n, p, C, AC);  
    add_matrices(m, p, AB, AC, RHS);
```

```
    printf("\nLHS (A * (B + C)):\n"); print_matrix(m, p, LHS);  
    printf("\nRHS ((A * B) + (A * C)):\n"); print_matrix(m, p, RHS);
```

```
    if (are_matrices_equal(m, p, LHS, RHS))  
        printf("\nVERIFIED: Left Distributive Property Holds True.\n");  
}
```

```
void verify_right_distributive(void) {  
    int m, n, p;  
    printf("Enter m, n, p: "); scanf("%d %d %d", &m, &n, &p);  
    int A[m][n], B[m][n], C[n][p], A_plus_B[m][n], LHS[m][p], AC[m][p],  
    BC[m][p], RHS[m][p];
```

```
    read_matrix(m, n, A, "A"); read_matrix(m, n, B, "B"); read_matrix(n, p, C, "C");
```

```
    add_matrices(m, n, A, B, A_plus_B); multiply_matrices(m, n, A_plus_B, n, p, C, LHS);  
    multiply_matrices(m, n, A, n, p, C, AC); multiply_matrices(m, n, B, n, p, C, BC);  
    add_matrices(m, p, AC, BC, RHS);
```

```
    printf("\nLHS ((A + B) * C):\n"); print_matrix(m, p, LHS);  
    printf("\nRHS ((A * C) + (B * C)):\n"); print_matrix(m, p, RHS);
```

```
    if (are_matrices_equal(m, p, LHS, RHS))  
        printf("\nVERIFIED: Right Distributive Property Holds True.\n");  
}
```

```
int main(void) {
```

```

int choice;
do {
    printf("\n1. Left Distributive\n2. Right Distributive\n3. Exit\nChoice: ");
    scanf("%d", &choice);
    switch (choice) {
        case 1: verify_left_distributive(); break;
        case 2: verify_right_distributive(); break;
        case 3: printf("Exiting...\n"); break;
    }
} while (choice != 3);
return 0;
}

```

4. Execution Screenshots

```

kali@kali:~/HDOOC_Day34
$ cd HDOOC_Day34
$ nano day34.c
$ gcc day34.c -o day34
$ ./day34

===== MATRIX DISTRIBUTIVE PROPERTIES DEMO =====
1. Verify Left Distributive Property: A * (B + C) = (A * B) + (A * C)
2. Verify Right Distributive Property: (A + B) * C = (A * C) + (B * C)
3. Exit
Enter your choice (1-3): 1

----- 1. Verify Left Distributive Property: A * (B + C) = (A * B) + (A * C) -----
Enter dimension n (rows of A): 2 2
Enter dimension n (cols of A / rows of B, C): Enter dimension p (cols of B, C): Enter elements of Matrix A (2x2):
A[0][0]: 1 2 3 4
A[0][1]: A[1][0]: A[1][1]: Enter elements of Matrix B (2x2):
B[0][0]: 2 0 1 3
B[0][1]: B[1][0]: B[1][1]: Enter elements of Matrix C (2x2):
C[0][0]: 1 1 0 2
C[0][1]: C[1][0]: C[1][1]:
----- Left Hand Side: A * (B + C) (2x2) -----
5      11
13     23

----- Right Hand Side: (A * B) + (A * C) (2x2) -----
5      11
13     23

VERIFIED: Left Distributive Property holds true: A * (B + C) = (A * B) + (A * C).

===== MATRIX DISTRIBUTIVE PROPERTIES DEMO =====
1. Verify Left Distributive Property: A * (B + C) = (A * B) + (A * C)
2. Verify Right Distributive Property: (A + B) * C = (A * C) + (B * C)
3. Exit
Enter your choice (1-3):

```